

Tutorial 08

ENVX2001 – Applied Statistical Methods

Semester 1

Welcome to week 8!

This week's lecture introduced you to variable selection and how to produce a more parsimonious model. We explored developing our models by fitting the full model, checking assumptions and deciding whether anything needed transformation. Once we were happy assumptions were met we could consider whether all of our predictors were useful in explaining the variation in our response variable.

In this tutorial, we will explore a new dataset and follow the steps of model selection to produce a model that explains a sufficient amount of variation, without being unnecessarily complex.

You will learn how to:

1. Check for collinearity and influential observations in a MLR model
2. Run partial F-tests to compare nested models
3. Implement stepwise regression using the `step()` function in R

Setting up for our analysis

Workflow for fitting parsimonious models

1. Model development

- Start with full model and check assumptions (e.g. normality, homoscedasticity, linearity, etc.)
- Look for additional issues (e.g. multicollinearity, outliers, etc.) → correlations, leverage plot, VIF plots and statistics
- Consider transformations (e.g. log, sqrt, etc.). → Response: if assumptions aren't met, transform the response variable → Predictors: it may help to explore transformations of skewed predictors before fitting the model to explore if there is a relationship present. Transforming predictors generally does not affect assumptions, but it can help to linearise relationships and make them easier to interpret.
- Following any transformations, test assumptions again.

2. Model selection

- Use VIF as an initial step to get rid of highly correlated predictors.
- Perform variable selection using backward elimination (good and fast), because:

- Using r^2 as a criterion is not recommended (it is not a good measure of model fit, only a good measure of variance explained).
- Using partial F-test is good, but slow.

Packages

This tutorial uses `tidyverse`, `readxl`, `moments`, `psych`, `performance` and `car`. Install any you are missing by running the following **in the console**:

CODE

```
install.packages(c("tidyverse", "readxl", "moments", "psych", "performance", "car"))
```

If a package is already installed, R will simply reinstall it — no harm done.

The data



Figure 1: Cute little Dipper (bird) standing on a rock by the water. Image source: markmedcalf via Adobe Stock.

Dippers are thrush-sized birds living mainly in the upper reaches of rivers, which feed on benthic invertebrates by probing the river beds with their beaks. The dataset in this exercise contains

data from a biological survey which examined the nature of the variables thought to influence the breeding of British dippers.

Twenty-two sites were included in the survey. Some of the variables have been transformed.

Data can be found in: **Dippers.csv**

The variables measured were:

- Altitude site altitude
- Hardness water hardness
- RiverSlope river-bed slope
- LogCadd the numbers of caddis fly larvae, transformed
- LogStone the numbers of stonefly larvae, transformed
- LogMay the numbers of mayfly larvae, transformed
- LogOther the numbers of all other invertebrates collected, transformed
- Br_Dens the number of breeding pairs of dippers per 10 km of river

In the analyses, the four invertebrate variables were transformed using a $\log(x + 1)$ transformation.

Br_Dens will be the response variable in our models.

Analysis

Model development

(i) Read in the data and investigate the relationship between each of your variables. Describe the relationships, supported by values (correlation coefficient) and plots.

CODE

```
# read in the data
dippers <- read.csv("data/Dippers.csv")

library(tidyverse)
glimpse(dippers) # check the data
```

OUTPUT

```
Rows: 22
Columns: 8
$ Altitude    <int> 259, 198, 251, 184, 145, 145, 198, 160, 251, 159, 160, 145,...
$ Hardness    <dbl> 12.20, 22.00, 26.30, 22.50, 29.50, 39.90, 42.80, 59.60, 69.00,...
$ RiverSlope  <dbl> 10.90, 14.70, 6.90, 4.60, 1.91, 5.00, 6.20, 14.30, 4.60, 3.00,...
$ LogCadd     <dbl> 2.303, 2.890, 3.784, 4.419, 3.219, 3.932, 3.664, 4.431, 3.700,...
$ LogStone    <dbl> 5.242, 4.344, 5.231, 5.242, 3.829, 4.898, 4.357, 6.337, 5.400,...
$ LogMay      <dbl> 0.000, 3.401, 5.826, 5.749, 5.509, 5.749, 5.371, 0.000, 4.800,...
$ LogOther    <dbl> 1.386, 1.609, 1.386, 1.386, 1.099, 3.045, 1.386, 2.944, 2.500,...
$ Br_Dens     <dbl> 3.60, 4.30, 3.80, 3.40, 3.80, 4.50, 4.30, 5.00, 4.50, 3.40,...
```

CODE

```
# Correlations and scatterplots all in one (+ histograms for each variable)
library(psych)
```

OUTPUT

Attaching package: 'psych'

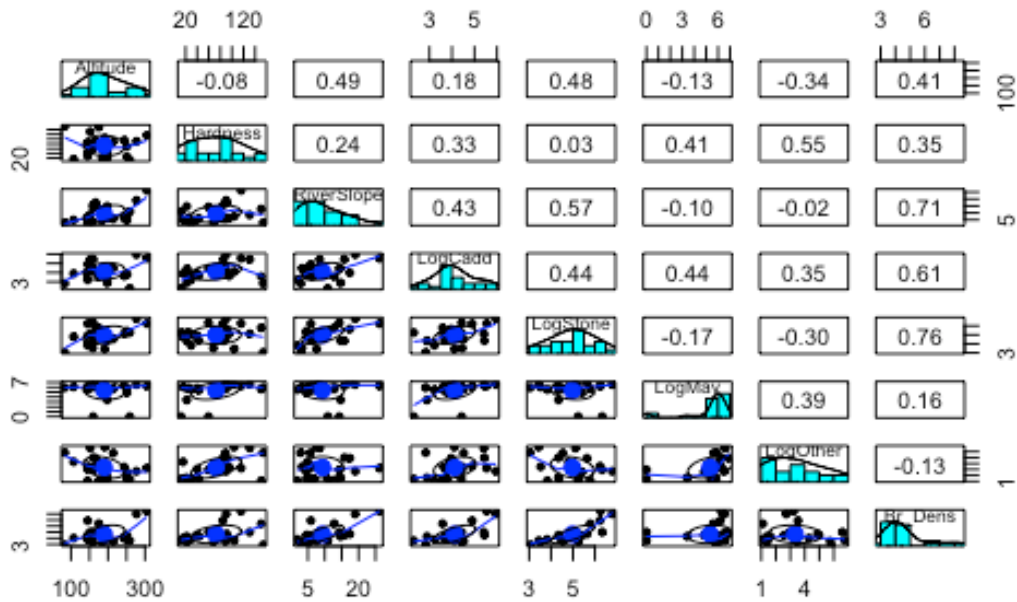
OUTPUT

The following objects are masked from 'package:ggplot2':

%%, alpha

CODE

```
pairs.panels(dippers)
```



CODE

```
# can also get the same results separately
cor(dippers)
```

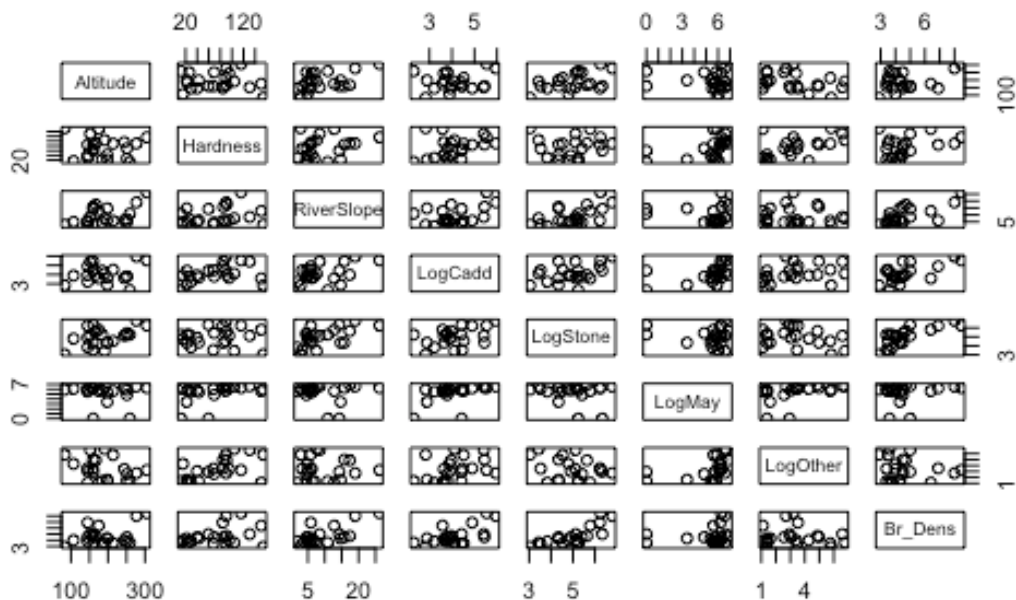
OUTPUT

	Altitude	Hardness	RiverSlope	LogCadd	LogStone
Altitude	1.00000000	-0.08170082	0.49003652	0.1807820	0.47736562
Hardness	-0.08170082	1.00000000	0.24208735	0.3337586	0.02951218
RiverSlope	0.49003652	0.24208735	1.00000000	0.4313081	0.57479242
LogCadd	0.18078205	0.33375857	0.43130806	1.00000000	0.44347440
LogStone	0.47736562	0.02951218	0.57479242	0.4434744	1.00000000
LogMay	-0.12782013	0.41356844	-0.09616686	0.4378978	-0.16744101

LogOther	-0.33859788	0.55399249	-0.02007183	0.3521440	-0.30025225
Br_Dens	0.40635349	0.35034817	0.71022164	0.6126891	0.76294367
	LogMay	LogOther	Br_Dens		
Altitude	-0.12782013	-0.33859788	0.4063535		
Hardness	0.41356844	0.55399249	0.3503482		
RiverSlope	-0.09616686	-0.02007183	0.7102216		
LogCadd	0.43789780	0.35214400	0.6126891		
LogStone	-0.16744101	-0.30025225	0.7629437		
LogMay	1.00000000	0.38820063	0.1562659		
LogOther	0.38820063	1.00000000	-0.1277591		
Br_Dens	0.15626585	-0.12775913	1.0000000		

CODE

```
pairs(dippers)
```



Between Br_Dens and other variables, there are a number of relatively strong correlations, particularly with LogStone ($r = 0.763$), RiverSlope ($r = 0.710$) and LogCadd ($r = 0.613$).

There are also some strong correlations between the predictor variables themselves, particularly between RiverSlope and LogStone ($r = 0.575$) and LogOther and Hardness ($r = 0.554$). This could be an issue for our model, as collinearity can make it difficult to interpret the model parameters and can inflate the standard errors of the estimates.

We can confirm if correlations between predictors may be an issue with the model by observing the Variance inflation factor (VIF) later on when we check our model assumptions.

(ii) Fit a multiple linear regression model to the data using all variables. Set Br_Dens as the response variable and the remaining variables as the predictors. This will be known as the 'full model'.

```
CODE
# fit the full model

full_model <- lm(Br_Dens ~ Altitude + Hardness + RiverSlope + LogCadd + LogStone + LogMay +
LogOther, data = dippers)
```

(iii) Check the assumptions of your model. Do you need to transform any variables? What does the Variance Inflation Factor (VIF) tell you about the collinearity between predictor variables?

The VIF will appear as one of the plots in the `check_model()` function, but it can also be helpful to get the values. You can use the `vif()` function from the `car` package to get the VIF values for each predictor variable.

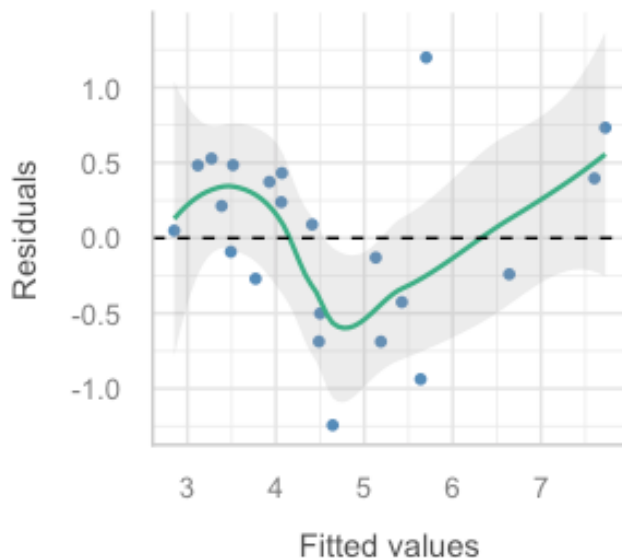
The `check_model` function has been squishing plots, so it may be better at times to plot them individually. You may use the following code, adapted to your model object names:

```
CODE
library(performance)

# Linearity
check_model(full_model, check = c("linearity"))
```

Linearity

Reference line should be flat and horizontal

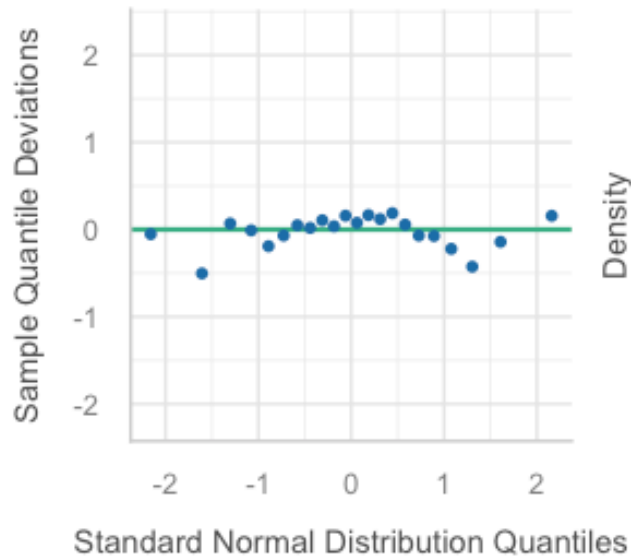


```
CODE
```

```
# Normality
check_model(full_model, check = c("qq", "normality"))
```

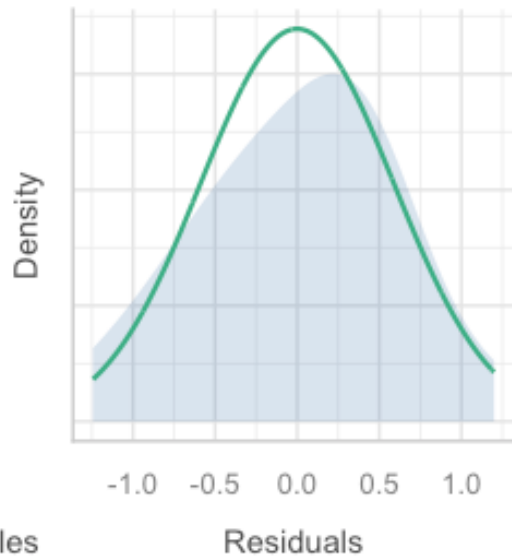
Normality of Residuals

Dots should fall along the line



Normality of Residuals

Distribution should be close to the nor

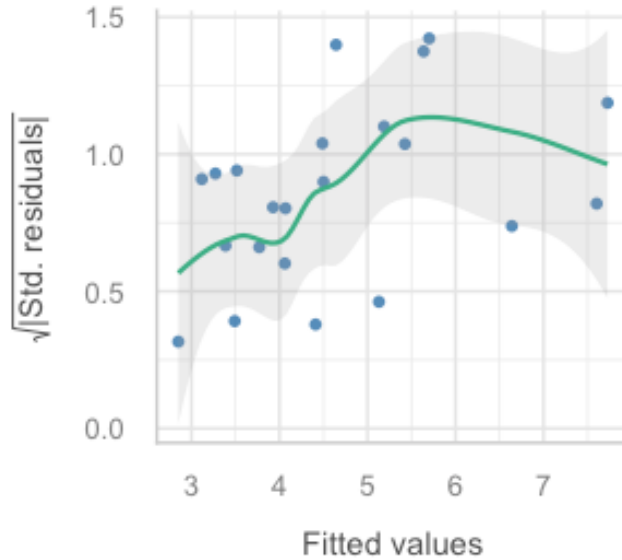


CODE

```
# Equal variance
check_model(full_model, check = c("homogeneity"))
```

Homogeneity of Variance

Reference line should be flat and horizontal

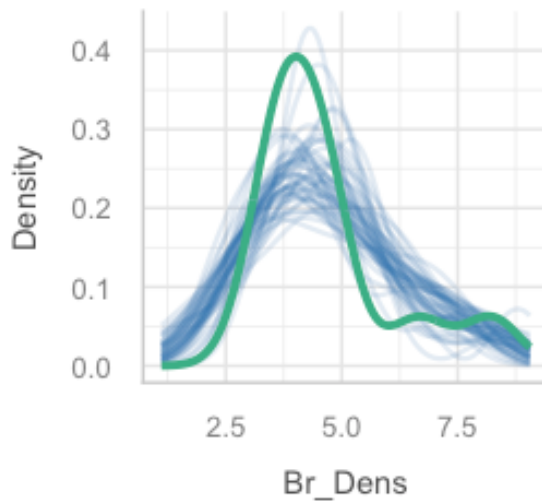


CODE

```
# Outliers
check_model(full_model, check = c("outliers", "pp_check"))
```

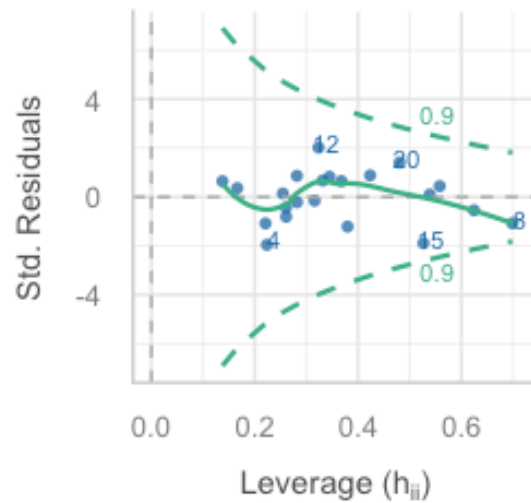
Posterior Predictive Check

Model-predicted lines should resemble observed data line



Influential Observations

Points should lie inside the contour line



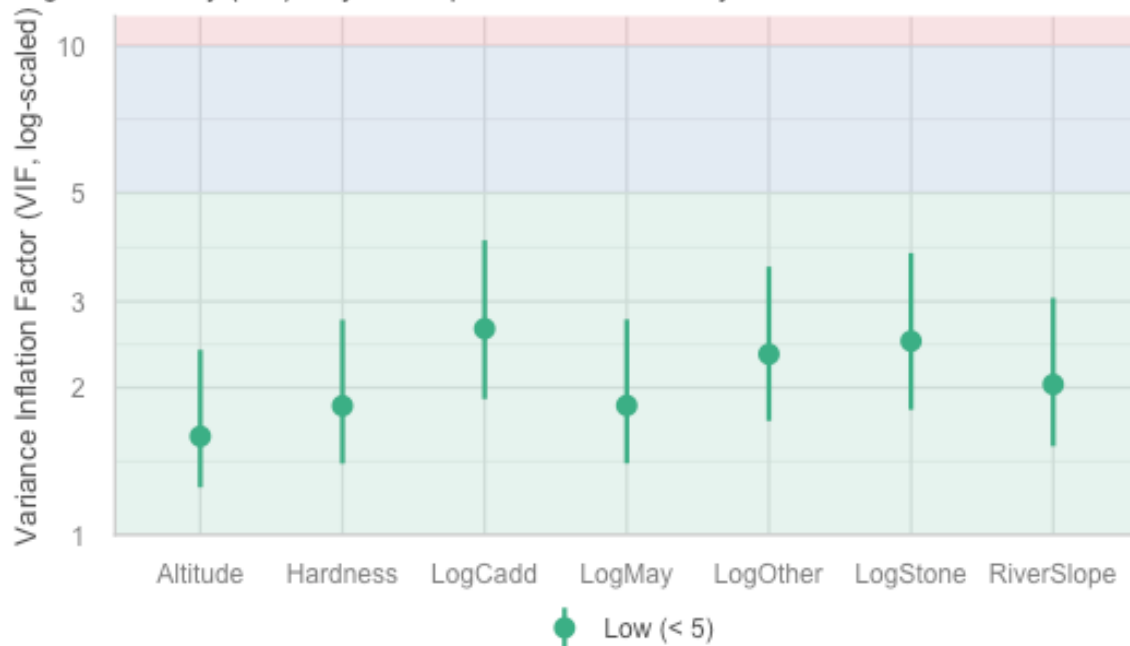
— Observed data — Model-predicted data

CODE

```
# VIF - visual
plot(performance::check_collinearity(full_model))
```

Collinearity

High collinearity (VIF) may inflate parameter uncertainty



CODE

```
# VIF - numbers
# comes from the car package
car::vif(full_model)
```

OUTPUT

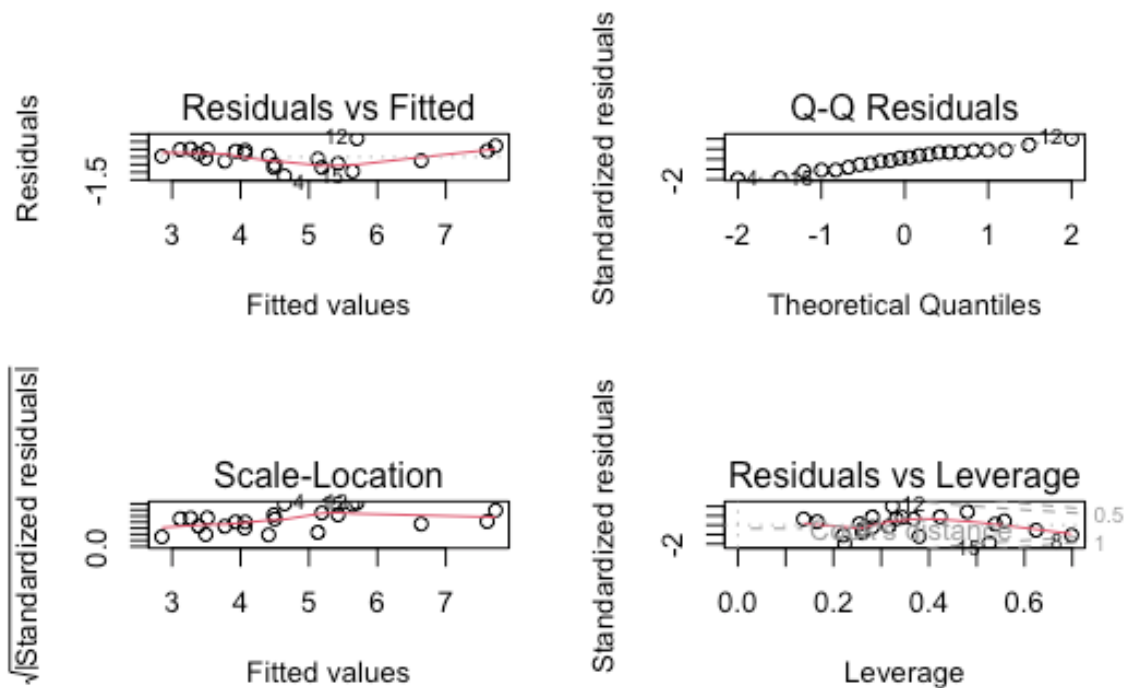
Altitude	Hardness	RiverSlope	LogCadd	LogStone	LogMay	LogOther
1.589862	1.835610	2.030493	2.639160	2.489586	1.838431	2.341119

CODE

```
#### ALTERNATIVE: plot() function to check assumptions

par(mfrow=c(2,2))

plot(full_model)
```



CODE

```
par(mfrow=c(1,1))
```

- **Linearity:** The relationship between the predictors and the response variable appears to be linear, as the residuals are mostly scattered around the horizontal line at zero in the residuals vs fitted plot.
- **Independence:** The residuals appear to be independent, as there is no clear pattern in the residuals vs fitted plot.
- **Normality:** The residuals appear to be approximately normally distributed, as the points in the Q-Q plot roughly follow a straight line. When we look at the `check_model()` version of the plot we see a slight n-shape to the points, but this is only small compared to the overall distribution of the points, so we are not too concerned about this.
- **Equal variance:** The residuals appear to have constant variance, as there is no clear pattern or fanning present in the standardised residuals vs fitted plot (scale-location in the base plot).
- **Leverage:** No single observations lying outside the Cook's distance line, indicating no influential observations.

VIF and collinearity

VIF Thresholds vary, depending on who you ask. General rule is that if it is greater than 10, then you need to remove the associated predictor. Some conservative people prefer a value of 5, so we will go with this.

Our threshold is 5, so given all of our VIFs are below this, we are not too concerned about collinearity. However, because they are close to the threshold it is something to keep in mind when interpreting the model parameters.

(iv) Make a note of the model parameters and fit statistics for the full model.

CODE

```
summary(full_model)
```

OUTPUT

Call:

```
lm(formula = Br_Dens ~ Altitude + Hardness + RiverSlope + LogCadd +  
    LogStone + LogMay + LogOther, data = dippers)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-1.24313	-0.38686	0.06925	0.42252	1.19902

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-0.881054	1.215055	-0.725	0.4803
Altitude	-0.001801	0.003537	-0.509	0.6186
Hardness	0.009956	0.004961	2.007	0.0645 .
RiverSlope	0.080254	0.035781	2.243	0.0416 *
LogCadd	0.408917	0.274412	1.490	0.1584
LogStone	0.604083	0.249207	2.424	0.0295 *
LogMay	0.114607	0.114116	1.004	0.3323
LogOther	-0.267073	0.131828	-2.026	0.0623 .

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.7208 on 14 degrees of freedom

Multiple R-squared: 0.8433, Adjusted R-squared: 0.765

F-statistic: 10.77 on 7 and 14 DF, p-value: 0.0001085

We can see that there are two significant predictors (RiverSlope and LogStone) in the full model, but there are also a number of non-significant predictors. The adjusted R^2 value is 0.77, which indicates that the model explains 77% of the variation in the response variable.

Because there are so many non-significant predictors in the model, we may want to consider removing some of them to produce a more parsimonious model.

(v) Select the least significant predictor and remove it from the model.

In this case, the least significant predictor is Altitude ($P = 0.619$). Remove it from the model and run the model again. This will be known as the 'reduced model'.

CODE

```
reduced_model <- lm(Br_Dens ~ Hardness + RiverSlope + LogCadd + LogStone + LogMay + LogOther, data  
= dippers)
```

(vi) Is the reduced model an improvement on the full model? How can you tell? What statistics would you look at to determine this?

CODE

```
summary(full_model)
```

OUTPUT

```
Call:
lm(formula = Br_Dens ~ Altitude + Hardness + RiverSlope + LogCadd +
    LogStone + LogMay + LogOther, data = dippers)

Residuals:
    Min       1Q   Median       3Q      Max
-1.24313 -0.38686  0.06925  0.42252  1.19902

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) -0.881054   1.215055  -0.725   0.4803
Altitude     -0.001801   0.003537  -0.509   0.6186
Hardness      0.009956   0.004961   2.007   0.0645 .
RiverSlope    0.080254   0.035781   2.243   0.0416 *
LogCadd       0.408917   0.274412   1.490   0.1584
LogStone      0.604083   0.249207   2.424   0.0295 *
LogMay       0.114607   0.114116   1.004   0.3323
LogOther     -0.267073   0.131828  -2.026   0.0623 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.7208 on 14 degrees of freedom
Multiple R-squared:  0.8433,    Adjusted R-squared:  0.765
F-statistic: 10.77 on 7 and 14 DF,  p-value: 0.0001085
```

CODE

```
summary(reduced_model)
```

OUTPUT

```
Call:
lm(formula = Br_Dens ~ Hardness + RiverSlope + LogCadd + LogStone +
    LogMay + LogOther, data = dippers)

Residuals:
    Min       1Q   Median       3Q      Max
-1.21925 -0.36896  0.03995  0.40149  1.27830

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) -1.112573   1.098551  -1.013   0.3272
Hardness      0.010101   0.004829   2.092   0.0539 .
RiverSlope    0.073829   0.032643   2.262   0.0390 *
LogCadd       0.405503   0.267470   1.516   0.1503
LogStone      0.587737   0.240949   2.439   0.0276 *
LogMay       0.111440   0.111096   1.003   0.3317
LogOther     -0.251481   0.125014  -2.012   0.0626 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.7028 on 15 degrees of freedom
Multiple R-squared:  0.8404,    Adjusted R-squared:  0.7766
F-statistic: 13.17 on 6 and 15 DF,  p-value: 3.139e-05
```

Removing the Altitude predictor from the model has slightly increased the adjusted R^2 value. This is an improvement in terms of parsimony, but we will need to use hypothesis testing to

determine whether the reduced model is a *significant* improvement on the full model. We can do this using an F-test.

Checkpoint

You should now be able to prepare an appropriate model by refining it, checking your assumptions, and ensuring collinearity is not an issue.

Model selection with F-tests

We will use the partial F-test to determine whether there is a significant difference between the models.

(i) State hypotheses

H0: no significant difference between the full and reduced models H1: There is a significant difference between the full and reduced models

If we fail to reject H0, then we would opt for the reduced model as it is more parsimonious (i.e. it is explaining a similar amount of variation with fewer predictors).

If we reject H0, then we would opt for the full model as it is a significant improvement on the reduced model.

(ii) Run the test

Once we have established our hypotheses, we can perform the F-test using the `anova()` function to compare the full and reduced models.

CODE

```
anova(full_model, reduced_model)
```

OUTPUT

Analysis of Variance Table

Model 1: Br_Dens ~ Altitude + Hardness + RiverSlope + LogCadd + LogStone + LogMay + LogOther

Model 2: Br_Dens ~ Hardness + RiverSlope + LogCadd + LogStone + LogMay + LogOther

	Res.Df	RSS	Df	Sum of Sq	F	Pr(>F)
1	14	7.2739				
2	15	7.4085	-1	-0.13464	0.2591	0.6186

(iii) Are the models significantly different?

The P-value is 0.6186, which is larger than 0.05, so we fail to reject the null hypothesis.

This means that the full model is not a significant improvement on the reduced model, and we would opt for the reduced model as it is more parsimonious.

i Checkpoint

You should now be able to use an F-test to determine whether the reduced model is a significant improvement on the full model.

Remember: the F-test is only useful if comparing nested models.

Model selection using Stepwise regression

When we have many predictors, this manual testing can become time consuming, so we can choose to use the `step()` function in R, which performs stepwise regression. This function will automatically remove the least significant predictor and test the model again, until it finds the best model based on the AIC criterion.

(i) Run the `step()` function on the full model to perform stepwise regression.

CODE

```
step_model <- step(full_model, direction = "backward")
```

OUTPUT

Start: AIC=-8.35

Br_Dens ~ Altitude + Hardness + RiverSlope + LogCadd + LogStone +
LogMay + LogOther

	Df	Sum of Sq	RSS	AIC
- Altitude	1	0.13464	7.4085	-9.9451
- LogMay	1	0.52404	7.7979	-8.8181
<none>			7.2739	-8.3486
- LogCadd	1	1.15372	8.4276	-7.1097
- Hardness	1	2.09230	9.3662	-4.7866
- LogOther	1	2.13246	9.4063	-4.6925
- RiverSlope	1	2.61383	9.8877	-3.5945
- LogStone	1	3.05288	10.3268	-2.6387

Step: AIC=-9.95

Br_Dens ~ Hardness + RiverSlope + LogCadd + LogStone + LogMay +
LogOther

	Df	Sum of Sq	RSS	AIC
- LogMay	1	0.49696	7.9055	-10.5167
<none>			7.4085	-9.9451
- LogCadd	1	1.13522	8.5437	-8.8086
- LogOther	1	1.99863	9.4071	-6.6906
- Hardness	1	2.16061	9.5691	-6.3150
- RiverSlope	1	2.52643	9.9349	-5.4897
- LogStone	1	2.93869	10.3472	-4.5952

Step: AIC=-10.52

Br_Dens ~ Hardness + RiverSlope + LogCadd + LogStone + LogOther

	Df	Sum of Sq	RSS	AIC
<none>			7.9055	-10.5167
- RiverSlope	1	2.1227	10.0282	-7.2842
- LogOther	1	2.3128	10.2182	-6.8712
- LogStone	1	2.4742	10.3797	-6.5262
- LogCadd	1	2.9091	10.8146	-5.6232
- Hardness	1	3.3747	11.2801	-4.6960

(ii) What is the final model that the `step()` function has produced? How does it compare to the full and reduced models?

Since we have stored the `step()` output as an object, we can just call the object, which will show us the final model:

```
CODE
step_model
```

OUTPUT

```
Call:
lm(formula = Br_Dens ~ Hardness + RiverSlope + LogCadd + LogStone +
    LogOther, data = dippers)

Coefficients:
(Intercept)      Hardness    RiverSlope      LogCadd      LogStone      LogOther
   -0.72462      0.01181      0.06538      0.54880      0.51249     -0.26813
```

```
CODE
## OR

summary(step_model)
```

OUTPUT

```
Call:
lm(formula = Br_Dens ~ Hardness + RiverSlope + LogCadd + LogStone +
    LogOther, data = dippers)

Residuals:
    Min       1Q   Median       3Q      Max
-1.18190 -0.49120 -0.08373  0.41721  1.31073

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) -0.724619   1.028416  -0.705   0.4912
Hardness     0.011811   0.004519   2.613   0.0188 *
RiverSlope   0.065384   0.031545   2.073   0.0547 .
LogCadd      0.548799   0.226169   2.426   0.0274 *
LogStone     0.512494   0.229020   2.238   0.0398 *
LogOther    -0.268128   0.123932  -2.164   0.0460 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.7029 on 16 degrees of freedom
Multiple R-squared:  0.8297,    Adjusted R-squared:  0.7765
F-statistic: 15.59 on 5 and 16 DF,  p-value: 1.169e-05
```

We can see that the selected model is:

Br_Dens ~ Hardness + RiverSlope + LogCadd + LogStone + LogOther

Or with the addition of the partial regression coefficients, our equation is:

Br_Dens = $-0.725 + 0.012 * \text{Hardness} + 0.065 * \text{RiverSlope} + 0.549 * \text{LogCadd} + 0.512 * \text{LogStone} - 0.268 * \text{LogOther}$

We can see there has also been a marginal change in the adjusted R^2 value, which is now 0.7765, compared to 0.765 for the full model and 0.7766 for the reduced model. This supports the idea that the full model explained a similar amount of variation in the response compared to the simplest model obtained from the `step()` function. Therefore it makes sense to select the simpler model.

(iii) How does the AIC of the stepwise regression model compare to the full and reduced models?

We can test this using the `AIC()` function:

CODE

```
AIC(full_model, reduced_model, step_model)
```

OUTPUT

	df	AIC
full_model	9	56.08470
reduced_model	8	54.48820
step_model	7	53.91655

Full_model AIC = 56.1

Reduced_model AIC = 54.5

step_model AIC = 53.9

As we select the model with the smallest AIC, we can confirm that the model selected by the `step()` function is better.

You may notice that the AIC values are different between the `step()` function and the `AIC()` function. This is because the `step()` function uses a different method to calculate the AIC values, which is based on the likelihood of the model, while the `AIC()` function uses a different method based on the residual sum of squares.

i Checkpoint

You should now be able to run the `step()` function to select the most appropriate model.

Wrap-up

In this tutorial, we used the dippers dataset to get some practice testing model assumptions and checking for collinearity. We then used partial F-tests and stepwise regression to select a more parsimonious model.

Example exam questions

1. What assumptions do we need to check for when fitting a multiple linear regression?
2. Why don't we just rely on the adjusted r^2 to compare models?
3. Can F-tests be run on non-nested models? Why or why not?